Vel Tech Group of Institutions

Name: Gontu Yaswanth
Email: vtu33434@veltech.edu.in
Roll no: Vtu33434
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 11_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an integer n, representing the total number of stairs, the task is to determine the number of distinct ways to reach the top when at each move you can climb 1, 2, or 3 steps.

Write a program using the Dynamic Programming (Bottom-Up Approach) algorithm to ensure that the total number of possible ways to climb the stairs is correctly calculated and displayed.

Examples:

Input: 4

Output: 7

Explanation: There are seven ways: {1, 1, 1, 1}, {1, 2, 1}, {2, 1, 1}, {1, 1, 2}, {2,

2}, {3, 1}, {1, 3}.

Input: 3

Output: 4

Explanation: There are four ways: {1, 1, 1}, {1, 2}, {2, 1}, {3}.

Input Format

The input contains a single integer n, representing the total number of stairs.

Output Format

The output prints a single integer representing the total number of distinct ways to reach the top.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Output: 7

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int dp[25];

    // base cases
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 2;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2] + dp[i-3];
    }
```

```
    printf("%d", dp[n]);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Given a non-negative integer n, the task is to compute the n-th Lucas number using the Dynamic Programming (Bottom-Up Approach) algorithm. The Lucas sequence follows the recurrence relation:

L(0) = 2, L(1) = 1, and L(n) = L(n−1) + L(n−2) for n ≥ 2.

Write a program to ensure that the n-th Lucas number is correctly calculated and displayed.

Input Format

The input contains a single integer n, representing the position in the Lucas sequence.

Output Format

The output prints a single integer representing the n-th Lucas number.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
Output: 4

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    int dp[25];

    // base cases
    dp[0] = 2;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }

    printf("%d", dp[n]);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement:

Dr. Ananya is a computational biologist working on analyzing DNA sequences. She needs to identify the longest palindromic subsequence within a DNA string, which may represent certain symmetric patterns important for genetic research.

Given a string S representing a DNA sequence (composed of letters A, C, G, T), find the length of the longest subsequence of S that reads the same forwards and backwards.

Note:

A subsequence is formed by deleting zero or more characters without changing the order of the remaining characters.The DNA sequences can be long, and an efficient solution using dynamic programming is required to handle them within computational limits.

Input Format

The input displays a single line containing the DNA string S.

Output Format

The output displays a single integer representing the length of the longest palindromic subsequence in S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam

Output: 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int dp[105][105];

    // base case: single characters
    for (int i = 0; i < n; i++) {
        dp[i][i] = 1;
    }

    // fill table
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;

            if (s[i] == s[j]) {
```

```
            if (len == 2)
                dp[i][j] = 2;
            else
                dp[i][j] = 2 + dp[i+1][j-1];
        } else {
            dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
        }
    }
}

printf("%d", dp[0][n-1]);

return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Arjun is tracking the stock prices for the last N days. He wants to invest in a way that he buys and sells stocks alternately to maximize his profit. He can either buy or skip on a day, but after buying he must sell before buying again. Find the maximum profit he can achieve from the stock prices. Help him to implement the task.

Formula

Let dp[i][0] = maximum profit on day i if not holding a stock

Let dp[i][1] = maximum profit on day i if holding a stock

Transition:

dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]) either skip or sell today

dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]) either keep holding or buy today

Base case:

dp[0][0] = 0

dp[0][1] = -prices[0]

Result:

max_profit = dp[N-1][0]

Input Format

The first line of input consists of Integer N representing the number of days.

The second line of input consists of N Integers prices[i] representing the stock price on day i.

Output Format

The output prints: "Maximum profit:" max_profit

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: Maximum profit: 2

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int price[1005];

    for (int i = 0; i < n; i++) {
        scanf("%d", &price[i]);
    }
```

```
    int dp0 = 0;            // not holding
    int dp1 = -price[0];    // holding

    for (int i = 1; i < n; i++) {
        int new_dp0 = max(dp0, dp1 + price[i]);
        int new_dp1 = max(dp1, dp0 - price[i]);

        dp0 = new_dp0;
        dp1 = new_dp1;
    }

    printf("Maximum profit: %d", dp0);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statment

To celebrate Diwali, Dhoni wants to buy a variety of crackers. He noticed four different types of crackers on the menu list, each with a price. He's now trying to figure out how many different ways he can get the crackers for Rs.N.

Rockets - 1 RsSparklers - 2 RsChakkars - 3 RsFlower pots - 5 Rs

Example

Input

4

Output

The total number of ways to get crackers is 4

Explanation

1st way -> 4 rockets

2nd way -> 2 Sparklers

3rd way -> 1 Sparkler, 2 rockets

4th way -> 1 Chakkar, 1 rocket

Input Format

The input consists of a single integer N, representing the total amount (in rupees) Dhoni wants to spend on crackers.

Output Format

The output prints "The total number of ways to get crackers is X" Where X representing the total number of different ways Dhoni can buy crackers such that the total cost equals N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Output: The total number of ways to get crackers is 6

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int coins[] = {1, 2, 3, 5};
    long long dp[1000005] = {0};

    dp[0] = 1;

    for (int c = 0; c < 4; c++) {
        for (int i = coins[c]; i <= n; i++) {
            dp[i] += dp[i - coins[c]];
        }
    }

    printf("The total number of ways to get crackers is %lld", dp[n]);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***